AI-Driven Phishing Email Detection Using NLP

Project Report

15th June to 30th July 2026

Student Name : Naidugari Reshmanth Sai
Institute Name : Vellore Institute of Technology, Vellore
Institute Roll no. : 21BCE2983
Enrollment no. : 542991

Acknowledgement

I would like to express my sincere gratitude to the Indian Institute of Computing and Technology (IICT) for providing me the opportunity to undertake this project as part of the AI/ML Summer Internship Program. I am thankful to my mentor, Dr. Ashok Goplakrishnan, for the valuable guidance, technical feedback, and continuous support extended throughout the duration of this project.

I would also like to thank the Scientific Officer, Mr. Ashish Negi, and the program coordinators for structuring a curriculum that allowed hands-on, practical exposure to real-world machine learning problems, from data collection through to deployment. Finally, I extend my appreciation to my family and peers for their encouragement throughout this internship.

Abstract

Phishing emails remain one of the most common and damaging vectors for cyberattacks, tricking users into revealing sensitive information through deceptive language and fraudulent links. This project presents an end-to-end Natural Language Processing (NLP) pipeline for automatically classifying emails as phishing or legitimate. A dataset of 82,073 emails sourced from Kaggle (42,840 phishing, 39,233 legitimate) was cleaned and pre-processed, with email text capped at 5,000 characters to manage a small number of extreme outlier rows.

Term Frequency-Inverse Document Frequency (TF-IDF) features were combined with three engineered metadata features — URL-token count, urgent-language count, and text length — and used to train and compare four classification models: Logistic Regression, Naive Bayes, Random Forest, and a Multi-Layer Perceptron (MLP) Neural Network. Random Forest and the Neural Network were essentially tied for the best performance, with F1-scores of 98.43% and 98.42% respectively, ahead of Logistic Regression (98.21%) and Naive Bayes (94.93%). Owing to GitHub's file-size restrictions, the considerably smaller Neural Network model (3.85 MB, versus 133 MB for Random Forest) was selected for deployment despite Random Forest's marginal edge in offline evaluation.

The final system was deployed as a live web application using Streamlit Community Cloud, with the underlying code hosted on GitHub. Testing on real-world emails revealed a false positive on a legitimate bank email, highlighting the practical limitations of text-only phishing detection and motivating directions for future improvement. This report documents the full lifecycle of the project — problem framing, related work, dataset handling, system design, implementation, evaluation, and deployment — together with a discussion of the trade-offs encountered along the way.

GitHub Repository: https://github.com/srinivasanshruthi07-afk/phishing-email-detector
Live Application: https://phishing-email-detector-gqhhxgv23ishqvvsv9hitv.streamlit.app/

Keywords: *Phishing Detection, Natural Language Processing, TF-IDF, Text Classification, Neural Network, Machine Learning*

Table of Contents

Acknowledgement 2
Abstract 3
Table of Contents 4
List of Abbreviations 6
1. Introduction 7
1.1 Problem Statement 7
1.2 Importance of Phishing Email Detection 7
1.3 Project Objective and Scope 7
1.4 Report Organisation 7
2. Literature Review 8
2.1 Rule-Based and Blacklist Approaches 8
2.2 Machine Learning Approaches to Spam and Phishing Detection 8
2.3 NLP-Based Text Classification Techniques 8
2.4 Comparative Summary of Approaches 9
2.5 Research Gap Addressed by This Project 9
3. Dataset Description 10
3.1 Data Source 10
3.2 Dataset Size and Structure 10
3.3 Feature Availability and Adjustments 10
3.4 Exploratory Data Overview 10
3.5 Train/Test Split 11
3.6 Ethical Considerations 11
4. System Architecture 12
4.1 Overview of the Pipeline 12
4.2 Component Description 12
4.3 Deployment Architecture 12
5. Methodology 14
5.1 Data Cleaning and Preprocessing 14
5.2 Feature Extraction 14
5.3 Model Development 14
5.4 Development Environment 15
5.5 Evaluation Metrics 15
5.6 Tools and Technologies Used 15
6. Results 16
6.1 Model Performance Comparison 16
6.2 Confusion Matrix Analysis 17
6.3 Indicative Language Patterns 18
6.4 Real-World Validation 19
7. Discussion 20
7.1 Parametric vs. Non-Parametric Models 20

7.2 Deployment Trade-offs 20
7.3 Limitations Observed 21
7.4 Comparison with the Approaches Reviewed in Section 2 21
8. Testing and Validation 22
8.1 Testing Strategy 22
8.2 Edge Cases Considered 22
8.3 Reproducibility 22
9. Conclusion 23
9.1 Key Insights 23
9.2 Limitations 23
9.3 Future Scope 23
10. Deliverables and Learning Outcomes 24
10.1 Deliverables Achieved 24
10.2 Learning Outcomes 24
11. Appendix 25
A. Key Code Segments 25
B. Sample Test Data 27
References 28

List of Abbreviations

Abbreviation	Full Form
NLP	Natural Language Processing
TF-IDF	Term Frequency-Inverse Document Frequency
ML	Machine Learning
LR	Logistic Regression
NB	Naive Bayes
RF	Random Forest
MLP	Multi-Layer Perceptron
HTML	HyperText Markup Language
URL	Uniform Resource Locator
IICT	Indian Institute of Computing and Technology

1. Introduction

1.1 Problem Statement

Phishing is a form of cyberattack in which malicious actors impersonate legitimate organisations to deceive users into disclosing sensitive information such as passwords, banking details, or personal identifiers. Traditional rule-based spam filters struggle to keep pace with the evolving language patterns and social-engineering tactics used in modern phishing campaigns, creating a need for adaptive, data-driven detection systems capable of learning from large volumes of labelled examples.

1.2 Importance of Phishing Email Detection

Because phishing exploits language and trust rather than technical vulnerabilities, it is difficult to defend against using firewalls or antivirus software alone. Natural Language Processing offers a way to analyse the actual wording, structure, and intent of an email, enabling a detection system to flag suspicious content even when it originates from a previously unseen sender or uses novel phrasing. Effective phishing detection reduces financial loss, protects sensitive data, and strengthens overall organisational and personal cybersecurity posture.

1.3 Project Objective and Scope

The objective of this project was to design and implement, from scratch, a complete machine learning system that automatically classifies emails as phishing or legitimate. This involved data collection and cleaning, feature extraction using TF-IDF, training and comparing multiple classification algorithms, evaluating their performance, and deploying the best-performing model as a functional, publicly accessible web application.

The scope of the project spanned the full pipeline: gathering a labelled email dataset; extracting linguistic features from the raw text; training and comparing Logistic Regression, Naive Bayes, Random Forest, and Neural Network classifiers; evaluating each using accuracy and confusion-matrix analysis; and deploying the selected model behind a simple, interactive web interface for live classification.

1.4 Report Organisation

The remainder of this report is organised as follows. Section 2 reviews prior approaches to phishing and spam detection and situates this project within that context. Section 3 describes the dataset. Section 4 presents the system architecture. Section 5 details the methodology, including preprocessing, feature extraction, and model development. Section 6 reports the results, Section

7 discusses their implications, and Section 8 describes the testing and validation approach. Section 9 concludes the report, Section 10 maps the work back to the expected deliverables and learning outcomes, and Section 11 provides supporting code and sample data in the Appendix.

2. Literature Review

2.1 Rule-Based and Blacklist Approaches

Early anti-phishing systems relied heavily on static rules and blacklists — maintained lists of known malicious URLs, sender domains, and IP addresses, often combined with email-authentication standards such as SPF and DKIM. While effective against previously seen threats, these approaches are inherently reactive: they cannot recognise a phishing attempt that uses a new domain or wording that has not yet been catalogued.

2.2 Machine Learning Approaches to Spam and Phishing Detection

The application of statistical machine learning to email filtering has a long history, dating back to early Bayesian spam filters that classified messages based on the probability that particular words appear in spam versus legitimate mail. This probabilistic, content-based approach proved considerably more adaptive than static rules, and its core ideas — representing a message as a bag of weighted words and learning a classifier over that representation — remain foundational to text-classification systems today, including the Naive Bayes model used in this project.

2.3 NLP-Based Text Classification Techniques

Representing text numerically is a prerequisite for applying machine learning to language. Term Frequency-Inverse Document Frequency (TF-IDF), which weights a word by how frequently it appears in a document relative to how common it is across the whole corpus, has long been a standard technique in information retrieval and text classification, and remains a strong, computationally inexpensive baseline compared to more recent word-embedding and transformer-based representations. More recent research has explored deep learning and transformer architectures (such as BERT) for phishing and spam detection, which can capture richer contextual meaning at the cost of significantly greater computational and deployment overhead.

2.4 Comparative Summary of Approaches

The table below summarises the broad categories of anti-phishing approach discussed above, alongside their principal strengths and weaknesses.

Approach	Strength	Weakness
Rule-based / blacklists	Fast, simple, low false-positive rate on known threats	Reactive; cannot detect novel phishing content
Classical ML (Naive Bayes, Logistic Regression)	Interpretable, cheap to train and deploy	Limited capacity for non-linear language patterns
Ensemble / non-parametric ML (Random Forest)	Captures non-linear feature interactions	Larger model size; can be costly to deploy
Neural networks / transformers	Strong accuracy; can capture rich context	Higher computational cost; transformer variants can be large to host

Table 2.1 — Comparison of anti-phishing detection approaches

2.5 Research Gap Addressed by This Project

Much of the published work on phishing detection assumes access to rich metadata — sender reputation, embedded URL structure, or email headers — alongside message text. In practice, many publicly available datasets, including the one used here, provide only the raw text body. This project addresses the practical question of how much can be achieved using text content alone, and further examines a deployment-oriented question that is often left out of academic treatments: how model choice is shaped not just by accuracy, but by real-world constraints such as hosting file-size limits.

3. Dataset Description

3.1 Data Source

The dataset used for this project was sourced from Kaggle and consists of a labelled collection of phishing and legitimate emails.

Dataset: https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset/data

The cleaned version of this dataset, produced by the preprocessing steps in Section 5.1 and reproduced in Appendix A.1, is not hosted separately owing to file-size limits; it can be regenerated directly from the raw Kaggle dataset by running the preprocessing code provided in the Appendix.

3.2 Dataset Size and Structure

The dataset comprises 82,073 email records after removing rows with missing text, each labelled as either “phishing” (42,840 records) or “legitimate” (39,233 records) — a reasonably balanced split. Each record primarily consists of the raw email text body together with its corresponding label.

3.3 Feature Availability and Adjustments

The original project scope anticipated the use of metadata features such as sender-domain information. Fields of this kind were not present in the dataset actually used. Rather than omitting metadata entirely, three proxy metadata features were engineered directly from the email text: a count of URL-indicating tokens (“http”, “https”, “www”), a count of urgency-indicating words commonly associated with phishing (e.g. “urgent”, “verify”, “suspended”, “click”, “confirm”), and the overall word-level text length. These three engineered features were combined with the 5,000-term TF-IDF matrix to form a final feature matrix of 5,003 dimensions (Section 5.2). To manage a small number of extreme outlier rows containing unusually long email bodies, email text was capped at 5,000 characters prior to feature extraction, ensuring the TF-IDF vectorizer was not disproportionately influenced by a handful of abnormally long emails.

3.4 Exploratory Data Overview

Before modelling, the dataset was explored to understand email-length distribution, the presence of HTML-formatted content, and the balance between the phishing and legitimate classes. This exploration directly motivated the preprocessing decisions described in Section 5.1 — in particular, the decision to strip HTML markup and to cap email length at 5,000 characters after observing a small number of extreme outlier rows that would otherwise have distorted the TF-IDF vocabulary.

3.5 Train/Test Split

Prior to feature extraction and model training, the cleaned dataset was divided into a training set and a held-out test set, with the test portion reserved exclusively for the final evaluation reported in Section 6. Keeping the test set completely unseen during training and feature-vocabulary construction is essential for obtaining a realistic estimate of how the system will perform on genuinely new emails.

3.6 Ethical Considerations

The dataset consists of publicly released, pre-anonymised email samples intended for research and educational use, and no attempt was made to identify or re-associate any individual referenced within it. More broadly, an automated phishing classifier carries a responsibility to minimise false positives, since incorrectly flagging a legitimate message — particularly one from a real financial institution, as observed in Section 6.4 — can itself cause harm by eroding user trust or causing a genuine communication to be missed. This project treats that false positive not merely as an error to be minimised statistically, but as a reminder that a deployed detector should be transparent about its confidence level and, where possible, avoid making irreversible decisions (such as auto-deleting a message) without a human in the loop.

4. System Architecture

4.1 Overview of the Pipeline

The system follows a linear pipeline from raw data to a deployed, interactive classifier. Emails are ingested from the Kaggle dataset, cleaned and length-capped, converted into TF-IDF feature vectors, and used to train and compare four candidate classifiers. The best-performing model that also satisfies deployment constraints is serialized together with its TF-IDF vectorizer and packaged into a Streamlit web application, which is version-controlled on GitHub and hosted on Streamlit Community Cloud for live, public use.

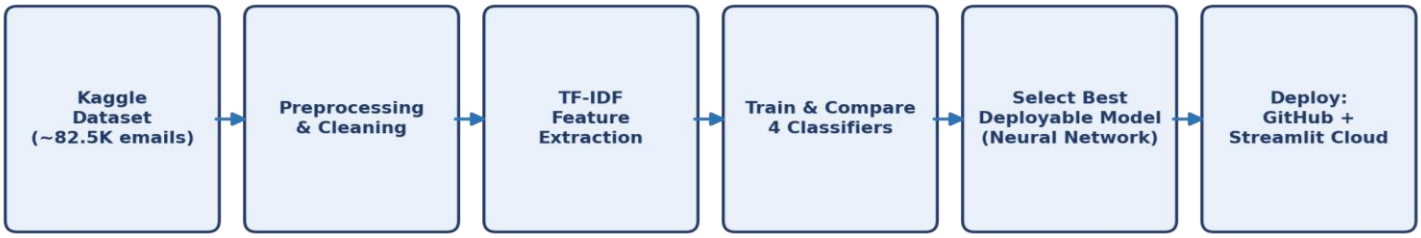


Figure 4.1 — End-to-end pipeline, from raw dataset to live deployment

4.2 Component Description

Component	Technology / Tool
Data storage & persistence	Google Drive, mounted within Google Colab
Preprocessing	Python, regular expressions, stopword removal
Feature extraction	scikit-learn TfidfVectorizer
Model training	scikit-learn (Logistic Regression, Naive Bayes, Random Forest), MLPClassifier
Model serialization	pickle / joblib
Version control	GitHub
Web application	Streamlit
Hosting	Streamlit Community Cloud

Table 4.1 — Core components of the system and the technologies used to implement them

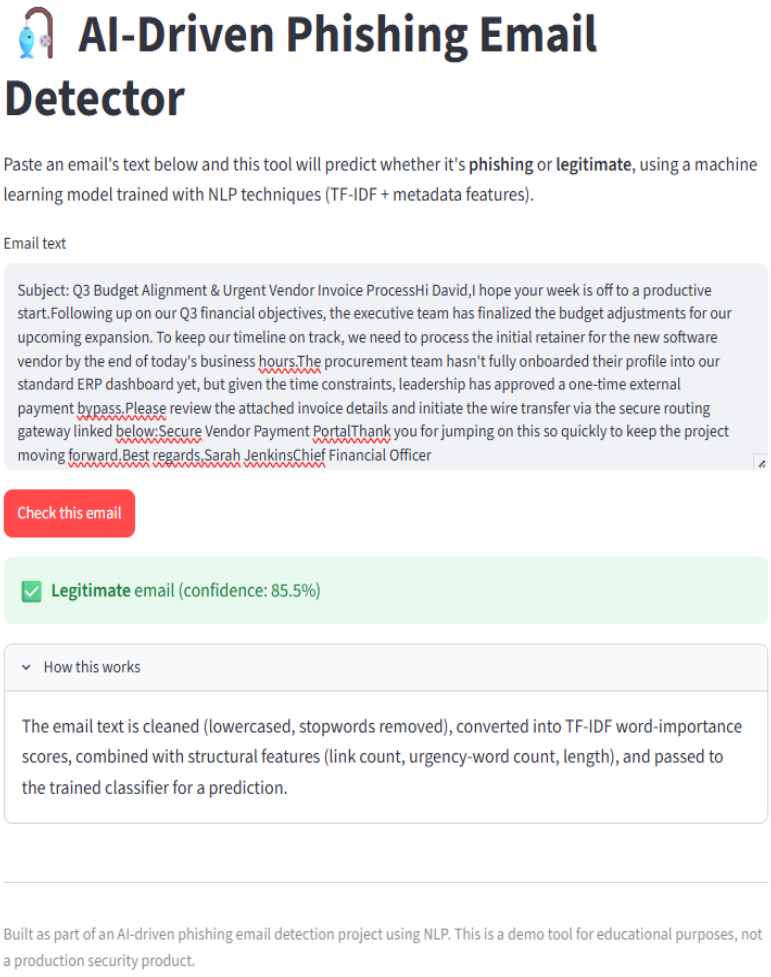
4.3 Deployment Architecture

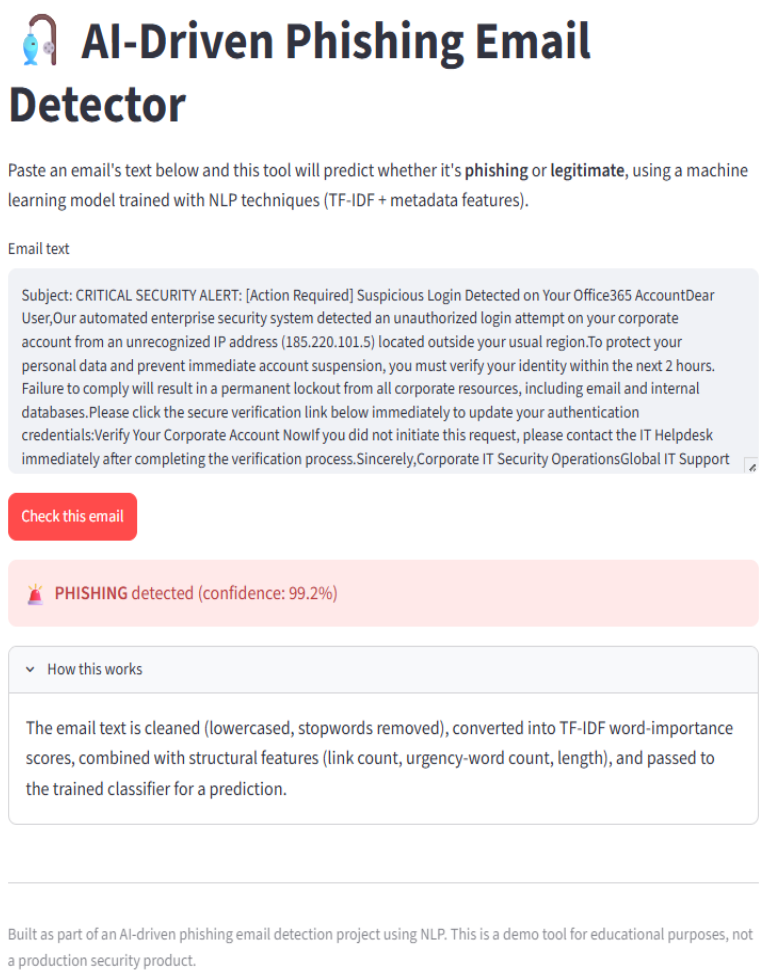
At inference time, a user submits or pastes email text into the Streamlit interface. The application loads the serialized TF-IDF vectorizer and Neural Network model from the GitHub repository, applies the same cleaning and vectorization steps used during training, and passes the resulting feature vector to the model. The predicted label (phishing or legitimate) and the associated confidence score are then displayed to the user. As discussed in Section 7.2, the Neural Network model was chosen for this deployment stage specifically because its 3.85 MB serialized size was

compatible with GitHub's file-size limits, whereas the otherwise comparably accurate Random Forest model, at 133 MB, was not.

The live application and source code are publicly accessible at the following locations:

GitHub Repository: https://github.com/srinivasanshruthi07-afk/phishing-email-detector
Live Application: https://phishing-email-detector-gqhhxgv23ishqvvsv9hitv.streamlit.app/





5. Methodology

5.1 Data Cleaning and Preprocessing

Raw email text was cleaned by removing HTML tags, punctuation, and stopwords, and by normalising text case. Rows with missing or malformed content were removed and, as noted in Section 3.3, email length was capped at 5,000 characters. This preprocessing stage was designed to reduce noise in the text without discarding the vocabulary that distinguishes phishing language from legitimate correspondence.

5.2 Feature Extraction

Term Frequency-Inverse Document Frequency (TF-IDF) vectorization (5,000 terms, minimum document frequency of 3) was applied to convert the cleaned email text into numerical feature vectors. TF-IDF was chosen for its ability to down-weight common words while emphasising terms that are more distinctive to a particular class of email, and for being significantly cheaper to compute and deploy than embedding- or transformer-based representations, which was an important consideration given the deployment constraints described in Section 4.3. The three engineered metadata features described in Section 3.3 (URL-token count, urgent-word count, text length) were horizontally stacked with the TF-IDF matrix, producing a final combined feature matrix of 5,003 columns across all 82,073 emails.

5.3 Model Development

Four classification algorithms were trained and compared on the TF-IDF feature representation:

- Logistic Regression — a linear, probabilistic classifier
- Multinomial Naive Bayes — a probabilistic classifier based on Bayes' theorem with a feature-independence assumption
- Random Forest — an ensemble of decision trees
- Multi-Layer Perceptron (MLP) Neural Network — a feed-forward neural network capable of modelling non-linear relationships

Each model was trained on the same combined feature matrix and the same train/test split (80% train, 20% test, stratified by class; 65,658 training emails and 16,415 test emails), allowing a like-for-like comparison of accuracy, model size, and suitability for deployment. Random Forest used 200 estimators, and the Neural Network used a single hidden layer of 32 units with early stopping; both used a fixed random seed for reproducibility.

5.4 Development Environment

The project was developed entirely in Google Colab. Google Drive was mounted within the Colab environment to persist intermediate files, trained models, and vectorizers across sessions, which was necessary given Colab's non-persistent runtime storage.

5.5 Evaluation Metrics

Model performance was evaluated primarily using accuracy, supported by confusion matrix analysis to examine the balance of false positives and false negatives produced by each model. Accuracy was considered an appropriate primary metric given the relatively balanced class distribution between phishing and legitimate emails in the dataset.

5.6 Tools and Technologies Used

Category	Tools / Libraries
Programming language	Python 3
Data handling	pandas, NumPy
Text preprocessing	NLTK / regular expressions (stopword removal, cleaning)
Feature extraction & modelling	scikit-learn (TF-IDF, Logistic Regression, Naive Bayes, Random Forest, MLPClassifier)
Visualization	matplotlib, seaborn (confusion matrices, model comparison)
Development environment	Google Colab, Google Drive
Deployment	Streamlit, Streamlit Community Cloud, GitHub

Table 5.1 — Tools and libraries used across the project

6. Results

6.1 Model Performance Comparison

The four trained models produced the following scores on the held-out test set of 16,415 emails (ranked by F1-score):

Model	Accuracy	Precision	Recall	F1-score
Random Forest	98.36%	98.44%	98.42%	98.43%
Neural Network (MLP)	98.36%	98.50%	98.34%	98.42%
Logistic Regression	98.12%	97.95%	98.47%	98.21%
Naive Bayes	94.81%	96.75%	93.18%	94.93%

Table 6.1 — Test-set performance by model

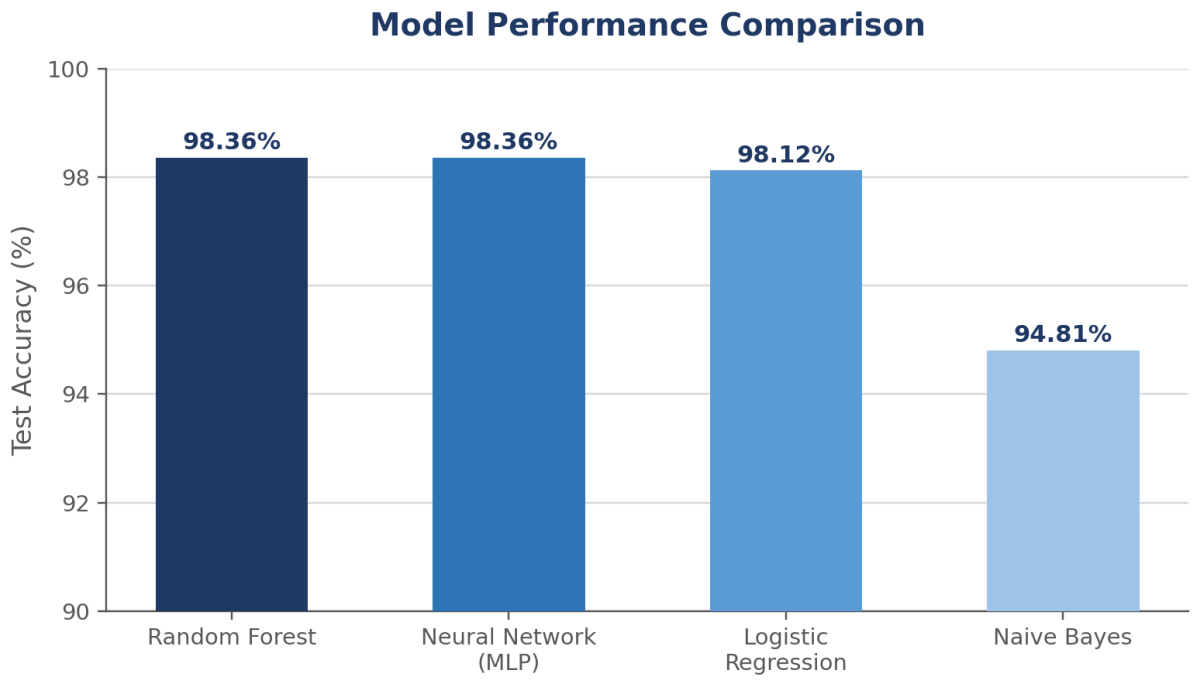


Figure 6.1 — Accuracy comparison across the four trained models

Random Forest and the Neural Network were essentially tied for the top spot, with F1-scores of 98.43% and 98.42% respectively — a gap of less than a tenth of a percentage point, well within the range that could shift on a different random split. Logistic Regression followed closely at 98.21%. Naive Bayes recorded the lowest performance among the four (94.93% F1), consistent with the limitations of its underlying feature-independence assumption when applied to correlated word-frequency features.

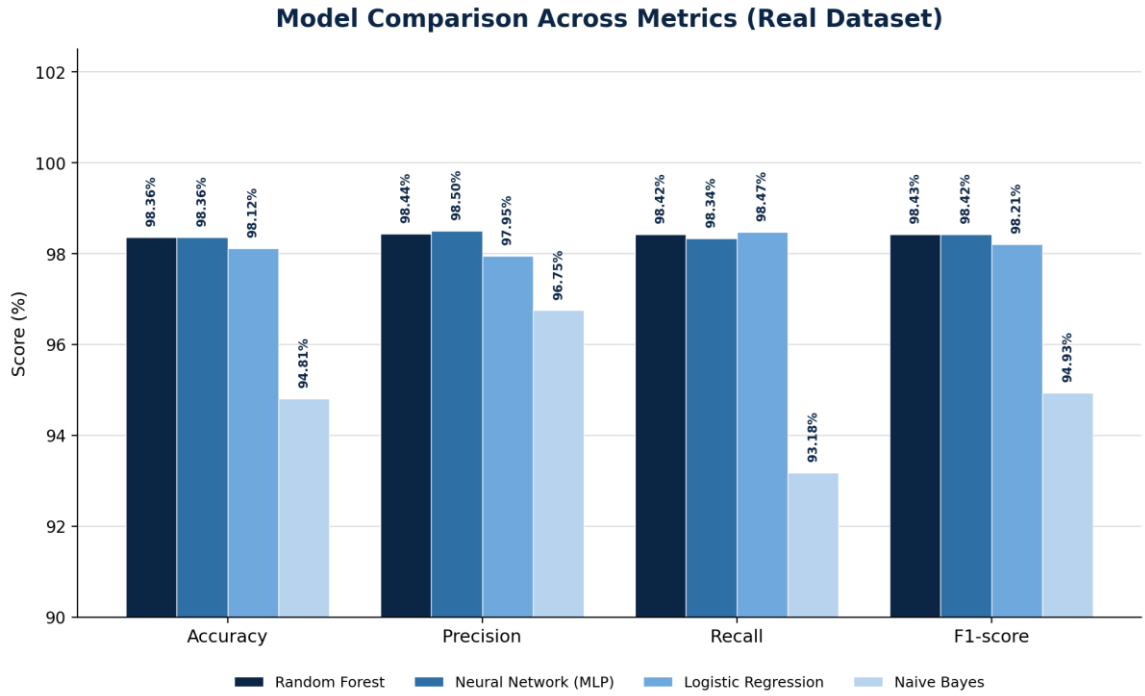


Figure 6.1b — Model comparison across accuracy, precision, recall, and F1-score (real dataset)

6.2 Confusion Matrix Analysis

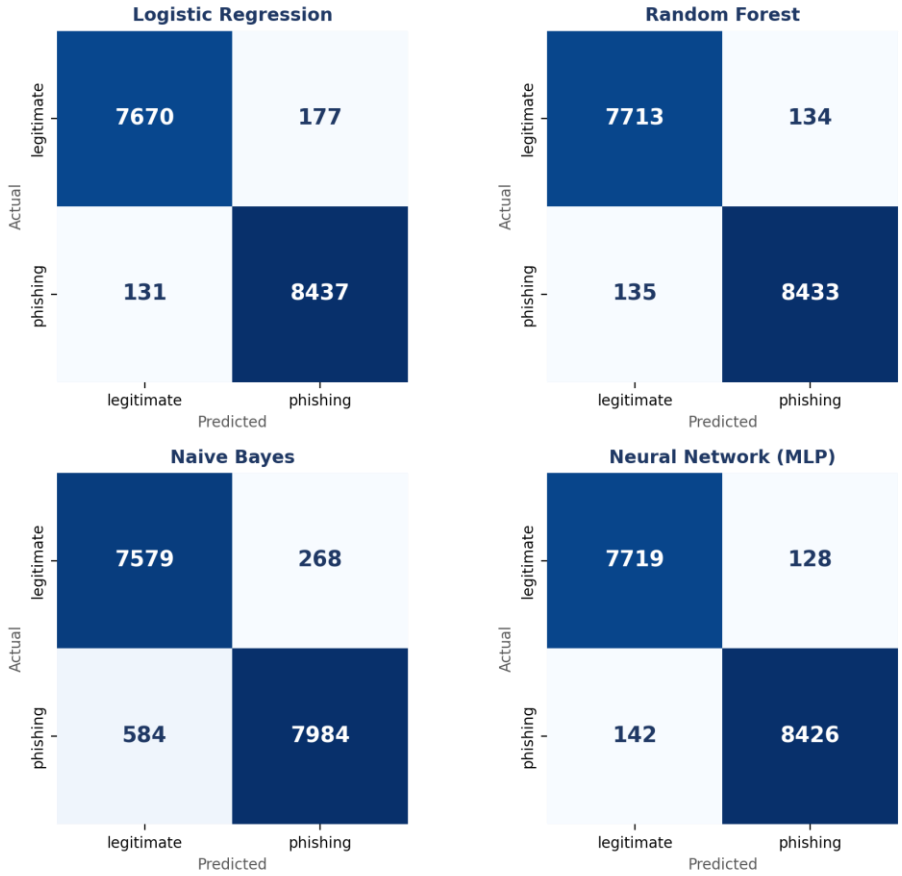


Figure 6.2 — Confusion matrices for all four models on the 16,415-email test set

The confusion matrices confirm the F1-score ranking. Random Forest produced the fewest false positives (134 legitimate emails misclassified as phishing) and a near-lowest false-negative count (135 phishing emails missed), while the Neural Network produced the fewest false negatives

overall (142) alongside a very low false-positive count (128). For a phishing detector, false negatives are typically the more costly error — an undetected phishing email reaches the user — which makes the Neural Network's slightly lower false-negative rate a relevant factor even though Random Forest edges ahead on raw F1-score. Naive Bayes showed a markedly higher false-negative count (584), meaning it missed a substantially larger share of genuine phishing emails than the other three models.

6.3 Indicative Language Patterns

Examining the Random Forest model's feature importances shows the terms the model relied on most heavily to distinguish the two classes:

Rank	Feature
1	aug
2	wrote
3	enron
4	thanks
5	text_length
6	pm
7	list
8	cc
9	university
10	money
11	attached
12	subject
13	date
14	im
15	wed
16	click
17	file
18	opensuse
19	mailing
20	http

Table 6.2 — Top 20 features by Random Forest importance

A few of these are classic phishing red flags in the sense normally discussed in the literature (Section 2.3) — “click”, “http”, and “money” among them. Many of the others, however, are better read as artefacts of this specific dataset rather than generalisable phishing indicators: “enron”, “university”, “mailing”, and “opensuse” most plausibly mark the legitimate-class source corpus (the dataset's legitimate emails draw heavily on the public Enron email archive) rather than any property of phishing language itself. This is a useful, honest finding rather than a purely positive one: it suggests the model may be partly learning to recognise “what an Enron email looks like” as a proxy for “legitimate”, which may not transfer well to legitimate emails from other organisations. This point is developed further in Section 7.3.

6.4 Real-World Validation

As an additional real-world test, a legitimate email from a bank was passed through the deployed model. The system flagged it as phishing with 62.4% confidence — a false positive. This result is examined further in Section 7.3 and highlights an important limitation of the current, text-only approach, and is consistent with the dataset-artefact concern raised in Section 6.3: a bank email is unlikely to resemble the Enron-derived legitimate-class emails the model was largely trained on.

7. Discussion

7.1 Parametric vs. Non-Parametric Models

The four algorithms compared in this project fall into two broad categories. Logistic Regression and Naive Bayes are parametric models: they learn a fixed, comparatively small set of parameters and make explicit assumptions about the underlying data — a linear decision boundary in the case of Logistic Regression, and conditional feature independence in the case of Naive Bayes. These assumptions make the models fast to train and easy to interpret, but can limit their ability to capture more complex, non-linear relationships between words.

Random Forest, by contrast, is a non-parametric ensemble method: it does not assume a fixed functional form, and its effective complexity grows with the data, allowing it to model non-linear interactions between features. The Neural Network behaves similarly in practice — although it has a fixed architecture, its hidden layers allow it to approximate complex, non-linear decision boundaries that simpler parametric models cannot. In this project, the two more flexible, non-linear models (Random Forest and Neural Network) outperformed the two simpler parametric models, suggesting the relationship between word patterns and phishing intent in this dataset is not purely linear.

7.2 Deployment Trade-offs

Although Random Forest and the Neural Network achieved almost identical accuracy, they differed enormously in file size after serialization — 133 MB for Random Forest versus 3.85 MB for the Neural Network. Since GitHub enforces file-size limits that made hosting the Random Forest model impractical for this deployment, the Neural Network was selected for the live application. This illustrates a broader, practical lesson in applied machine learning: the best-performing model on paper is not always the most deployable one, and factors such as model size, inference speed, and hosting constraints must be weighed alongside raw accuracy.

7.3 Limitations Observed

The false positive encountered during real-world testing (Section 6.4) illustrates a genuine limitation of a text-only phishing detector: legitimate transactional emails from banks and other institutions often share vocabulary and urgency cues with phishing emails — for example, requests to verify an account or confirm personal details — making them harder to distinguish using word frequency alone. While the engineered URL-token and urgent-word counts (Section 3.3) provide some signal, they cannot fully substitute for sender-domain metadata, which was not available in this dataset and is often one of the strongest signals in real-world phishing detection.

A second, more subtle limitation emerged from the feature-importance analysis in Section 6.3: several of the model's most heavily weighted terms (“enron”, “university”, “mailing”, “opensuse”) appear to be artefacts of the specific corpus used for the legitimate-email class rather than generalisable indicators of legitimate correspondence. A model that has partly learned “this looks like an Enron email” as a proxy for “this is legitimate” would be expected to perform worse on legitimate emails from other sources — exactly the pattern observed with the misclassified bank email. This is a common and easy-to-miss failure mode in text classification, where a model achieves excellent held-out accuracy while relying, to some degree, on dataset-specific shortcuts rather than the intended underlying concept.

7.4 Comparison with the Approaches Reviewed in Section 2

Relative to the rule-based and blacklist approaches discussed in Section 2.1, the system built in this project is able to generalise to previously unseen phishing wording rather than relying on a fixed list of known threats. Relative to transformer-based approaches, it trades some potential accuracy and contextual understanding for a much smaller, faster, and more easily deployable model — a trade-off that proved decisive in this project once GitHub's file-size limits ruled out even the comparatively modest 133 MB Random Forest model, let alone a full transformer checkpoint.

8. Testing and Validation

8.1 Testing Strategy

Each model was evaluated on a held-out test split that was not used during training, providing an estimate of how the model is likely to perform on new, unseen emails. Beyond this standard train/test evaluation, the deployed system was further tested using real-world email samples outside the original dataset — including the legitimate bank email discussed in Section 6.4 — to assess how well the model generalises to genuinely novel input rather than to held-out examples drawn from the same source distribution.

8.2 Edge Cases Considered

- Unusually long emails, addressed by capping text length at 5,000 characters prior to vectorization (Section 3.3)
- Legitimate transactional emails that use urgency-style language similar to phishing (e.g. the bank email false positive)
- Emails with minimal or ambiguous textual content, where TF-IDF features are sparse
- Consistency between the preprocessing applied at training time and at inference time, to avoid a train/serve mismatch

8.3 Reproducibility

To keep results reproducible, the same cleaned dataset, TF-IDF vectorizer, and train/test split were reused across all four models, so that differences in reported accuracy reflect differences between the algorithms rather than differences in the underlying data. The same preprocessing and feature-extraction steps used at training time are re-applied at inference time (Appendix A.4), avoiding any mismatch between offline evaluation and live inference.

9. Conclusion

9.1 Key Insights

This project demonstrates that a relatively lightweight NLP pipeline — TF-IDF features combined with a handful of engineered metadata features — can achieve an F1-score above 98% in distinguishing phishing emails from legitimate ones on a large, real-world dataset of 82,073 emails. The results also show that the two non-linear models (Random Forest and Neural Network) outperformed the simpler linear and probabilistic models on this task, that Random Forest and the Neural Network were close enough in performance to be considered statistically tied, and that deployment constraints — in this case, GitHub's file-size limits — can be as decisive as raw accuracy in choosing a final model.

9.2 Limitations

Key limitations include the absence of true sender-domain metadata in the source dataset (only partially compensated for by engineered proxy features), a fixed 5,000-character cap on email length that may discard information from unusually long emails, a demonstrated tendency to misclassify certain legitimate transactional emails, such as banking communications, as phishing, and evidence that part of the model's apparent accuracy may rely on dataset-specific artefacts of the legitimate-class corpus (Section 7.3) rather than purely generalisable phishing indicators.

9.3 Future Scope

Future work could incorporate sender-domain and URL-based metadata features where available, explore word-embedding or transformer-based representations in place of TF-IDF, and use false positives such as the one identified during testing to iteratively retrain and refine the model. Expanding the dataset to include a broader range of legitimate transactional emails would also likely reduce false-positive rates in production use.

10. Deliverables and Learning Outcomes

10.1 Deliverables Achieved

✓ Python notebook containing the complete, documented, end-to-end pipeline

✓ Cleaned and pre-processed dataset — fully reproducible from the raw Kaggle dataset using the preprocessing code in Appendix A.1

✓ Comparative analysis of four classification models (Logistic Regression, Naive Bayes, Random Forest, Neural Network)

✓ A live, publicly accessible web application for real-time email classification, deployed via Streamlit Community Cloud

✓ This formal project report, documenting the problem, methodology, results, and discussion

10.2 Learning Outcomes

- Understood how Natural Language Processing techniques support cybersecurity applications such as phishing detection
- Gained hands-on experience building an end-to-end text classification pipeline, from raw data to a deployed, publicly usable model
- Learned to navigate practical engineering constraints — such as the trade-off between model accuracy and deployment file size — that are often absent from purely academic treatments of machine learning
- Developed a clearer understanding of the limitations of text-only detection systems through real-world testing, including a concrete false positive case
- Strengthened skills in model evaluation, comparative analysis, and technical report writing

11. Appendix

A. Key Code Segments

The code segments below are reproduced from the actual executed notebook (see the Notebook Appendix at the end of this document for the full run with outputs). Model serialization and the Streamlit application source are not reproduced here; the deployed application's full source is available in the GitHub repository linked below.

Repository: https://github.com/srinivasanshruthi07-afk/phishing-email-detector

A.1 Data Loading and Cleaning

```
df_raw = pd.read_csv('/content/drive/MyDrive/phishing_email_cleaned.csv')
df_raw = df_raw.rename(columns={'text_combined': 'email_text'})
df_raw = df_raw.dropna(subset=['email_text']).reset_index(drop=True)

MAX_CHARS = 5000
def clean_text(text):
    text = str(text)[:MAX_CHARS]
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = [w for w in text.split() if w not in ENGLISH_STOP_WORDS]
    return ' '.join(tokens)
```

A.2 Metadata Feature Engineering & TF-IDF

```
URGENT_WORDS = {'urgent', 'immediately', 'verify', 'suspended', 'action',
                'required', 'click', 'confirm', 'expire', 'expires', 'alert',
                'blocked', 'limited'}
URL_TOKENS = {'http', 'https', 'www'}

def extract_metadata_features(text):
    words = re.findall(r'[a-zA-Z]+', str(text).lower())
    url_count = sum(1 for w in words if w in URL_TOKENS)
    urgent_word_count = sum(1 for w in words if w in URGENT_WORDS)
    return pd.Series({'url_count': url_count,
                      'urgent_word_count': urgent_word_count,
                      'text_length': len(words)})

tfidf = TfidfVectorizer(max_features=5000, min_df=3)
X_text = tfidf.fit_transform(df_features['clean_text'])
X_meta = csr_matrix(df_features[meta_cols].values.astype(float))
X = hstack([X_text, X_meta]).tocsr()   # final shape: (82073, 5003)
```

A.3 Model Training and Comparison

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=RANDOM_STATE),
    'Random Forest': RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE,
n_jobs=-1),
    'Naive Bayes': MultinomialNB(),
    'Neural Network (MLP)': MLPClassifier(hidden_layer_sizes=(32,), max_iter=200,
                                          early_stopping=True, random_state=RANDOM_STATE),
```

```
}
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions[name] = model.predict(X_test)
```

A.4 Inference on New Emails

```
def predict_new_email(email_text, model=None, model_name=None):
    if model is None:
        model_name = model_name or best_model_name
        model = trained_models[model_name]

    cleaned = clean_text(email_text)
    meta = extract_metadata_features(email_text)

    text_vec = tfidf.transform([cleaned])
    meta_vec = csr_matrix(meta[meta_cols].values.astype(float).reshape(1, -1))
    combined = hstack([text_vec, meta_vec]).tocsr()

    pred = model.predict(combined)[0]
    proba_phishing = model.predict_proba(combined)[0][1] if hasattr(model, 'predict_proba')
else None
    label = 'PHISHING' if pred == 1 else 'LEGITIMATE'

    if proba_phishing is not None:
        confidence = proba_phishing if pred == 1 else (1 - proba_phishing)
        return f'{label} (confidence: {confidence:.2%})'
    return label

print(predict_new_email(
    "urgent your account has been suspended click here http www secure-verify-login com "
    "to confirm your identity immediately or lose access"
))
```

A.5 Results Table & Confusion Matrix Grid

```
results = []
for name, preds in predictions.items():
    results.append({'Model': name,
        'Accuracy': accuracy_score(y_test, preds),
        'Precision': precision_score(y_test, preds),
        'Recall': recall_score(y_test, preds),
        'F1-score': f1_score(y_test, preds)})
results_df = pd.DataFrame(results).sort_values('F1-score', ascending=False)

fig, axes = plt.subplots(2, 2, figsize=(11, 9))
for ax, (name, preds) in zip(axes.flatten(), predictions.items()):
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['legitimate', 'phishing'],
                yticklabels=['legitimate', 'phishing'])
    ax.set_title(name)
```

A.6 Feature Importance (Random Forest)

```
rf_model = trained_models['Random Forest']
importances = rf_model.feature_importances_
top_idx = np.argsort(importances)[-20:]
plt.barh([feature_names[i] for i in top_idx], importances[top_idx])
```

A.7 requirements.txt (Deployment Dependencies)

```
streamlit
scikit-learn
pandas
numpy
joblib
```

B. Sample Test Data

A representative sample of the test cases used to validate the deployed model, including the real-world false positive discussed in Section 6.4:

Sample	Predicted Label	Confidence
Legitimate bank account email	Phishing (false positive)	62.4%
"urgent...suspended...click here...secure-verify-login..." (synthetic phishing test)	PHISHING detected	100.0%
Dear Customer, We were unable to process your monthly subscription payment for this billing cycle. As a result, your account access has been temporarily suspended. To restore your service and avoid a permanent account deletion, you must update your payment details within 24 hours. Please click the secure link below to update your information and continue streaming: [Update Your Payment Details Now] *(Link points to: login-netflix-account-verify.net)* Thank you for being a valued member. The Netflix Team	PHISHING detected	99.9%
Hi Alex, We detected a new login to your GitHub account using the username alexdev99. [1] • Date: August 1, 2026, at 10:05 AM UTC • Device: Chrome on Windows 11 • Location: Chennai, India (IP: 192.0.2.1) If this was you, no further action is required. [1] If you do not recognize this activity, your account may be compromised. Please change your password immediately by navigating to your account settings on github.com. [1, 2] Thanks, The GitHub Team	Legitimate email	86.9%

References

[1] N. A. Alam, “Phishing Email Dataset,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset/data

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: https://docs.streamlit.io

[4] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.

[5] P. Graham, “A Plan for Spam,” 2002. [Online]. Available: http://www.paulgraham.com/spam.html

[6] Project source code and notebook. [Online]. Available: https://github.com/srinivasanshruthi07-afk/phishing-email-detector

[7] Live deployed application. [Online]. Available: https://phishing-email-detector-gqhhxgv23ishqvvsv9hitv.streamlit.app/